

C Code

```
#include <stdio.h>

#include <stdlib.h>

#include <ctype.h>

#define MAX 100

struct Node {

    char data;

    struct Node *left;

    struct Node *right;

};

struct Node *stack[MAX];

int top = -1;

void push(struct Node *node) {

    stack[++top] = node;

}

struct Node *pop() {

    return stack[top--];

}

struct Node *createNode(char data) {

    struct Node *newNode = (struct Node *)malloc(sizeof(struct Node));

    newNode->data = data;

    newNode->left = NULL;

    newNode->right = NULL;

    return newNode;

}

int isOperator(char ch) {

    return ch == '+' || ch == '-' || ch == '*' || ch == '/';

}
```

```

}

struct Node *buildTree(char postfix[]) {

    top = -1;

    for (int i = 0; postfix[i] != '\0'; i++) {

        char ch = postfix[i];

        if (ch == ' ')

            continue;

        if (isdigit(ch)) {

            push(createNode(ch));

        }

        else if (isOperator(ch)) {

            struct Node *right = pop();

            struct Node *left = pop();

            struct Node *operatorNode = createNode(ch);

            operatorNode->left = left;

            operatorNode->right = right;

            push(operatorNode);

        }

    }

    return pop();

}

```

/* Traversals */

```

void preorder(struct Node *root) {

    if (root != NULL) {

        printf("%c ", root->data);

        preorder(root->left);

        preorder(root->right);

    }

}

```

```

    }
}

void inorder(struct Node *root) {
    if (root != NULL) {
        if (isOperator(root->data))
            printf("(");
        inorder(root->left);
        printf("%c ", root->data);
        inorder(root->right);
        if (isOperator(root->data))
            printf(")");
    }
}

void postorder(struct Node *root) {
    if (root != NULL) {
        postorder(root->left);
        postorder(root->right);
        printf("%c ", root->data);
    }
}

int evaluateTree(struct Node *root) {
    if (isdigit(root->data))
        return root->data - '0';

    int left = evaluateTree(root->left);
    int right = evaluateTree(root->right);
    switch (root->data) {
        case '+':

```

```

        return left + right;

    case '-':

        return left - right;

    case '*':

        return left * right;

    case '/':

        return left / right;

    }

    return 0;

}

int evaluatePostfix(char postfix[]) {

    int valueStack[MAX];

    int valueTop = -1;

    for (int i = 0; postfix[i] != '\0'; i++) {

        char ch = postfix[i];

        if (ch == ' ')

            continue;

        if (isdigit(ch)) {

            valueStack[++valueTop] = ch - '0';

        }

        else if (isOperator(ch)) {

            int right = valueStack[valueTop--];

            int left = valueStack[valueTop--];

            int result;

            switch (ch) {

                case '+':

                    result = left + right;

```

```

        break;

    case '-':

        result = left - right;

        break;

    case '*':

        result = left * right;

        break;

    case '/':

        result = left / right;

        break;

    }

    valueStack[++valueTop] = result;

}

}

return valueStack[valueTop];

}

int main() {

    char postfix[MAX];

    printf("Enter the postfix expression to be evaluated:\n");

    fgets(postfix, MAX, stdin);

    struct Node *root = buildTree(postfix);

    printf("\nExpression Tree Traversals\n");

    printf("Preorder  : ");

    preorder(root);

    printf("\nInorder   : ");

    inorder(root);

    printf("\nPostorder  : ");

```

```

postorder(root);

printf("\n\nEvaluation Results\n");

printf("Stack-based Postfix Evaluation = %d\n",

    evaluatePostfix(postfix));

printf("Expression Tree Evaluation    = %d\n",

    evaluateTree(root));

return 0;

}

```

Output

Enter the postfix expression to be evaluated:

842*62/-

Expression Tree Traversals

Preorder : - * 4 2 / 6 2

Inorder : ((4 * 2)- (6 / 2))

Postorder : 4 2 * 6 2 / -

Evaluation Results

Stack-based Postfix Evaluation = 5

Expression Tree Evaluation = 5